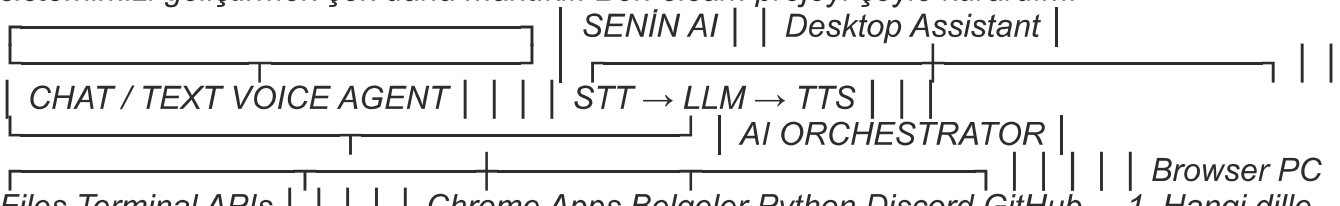
# Keke AI Asistan Proje Mimarisi

https://gemini.google.com/app/db6f2231f3fc81c2

*User prompt: bugün 14:23 keke önceki sohbette dediğim gibi ai gelistiriceğim ama bunu nasıl eğiticem hangi dille yazarsak daha iyi olur hangi entegrasyonları hangi modelleri hangi aileri kullansak daha iyi olur bana yardım et 👍 Aynen keke, önceki konuşmadaki hedefin bilgisayarda çalışan, sesli konuşabilen, chat arayüzü olan ve AI Agent gibi uygulamalara bağlanıp işlem yapabilen kendi AI asistanın. Burada en önemli nokta şu: İlk aşamada kendi LLM'imizi sıfırdan eğitmek yerine, güçlü bir hazır modeli "beyin" olarak kullanıp kendi agent/araç/hatırlama sistemimizi geliştirmek çok daha mantıklı. Ben olsam projeyi şöyle kurardım:*




*1. Hangi dille yazalım? 🐍 Ana dil: Python Ben Python seçerdim. Çünkü senin yapmak istediğin şeyde Python'un avantajı çok büyük: AI/LLM entegrasyonları Agent sistemleri API'ler bilgisayarla etkileşim otomasyon dosya işlemleri web scraping Python script çalıştırma ses işleme RAG embedding/vector database model entegrasyonları çok rahat. Örneğin AI şöyle bir karar verebilir: Kullanıcı: "Chrome'u aç ve GitHub'a gir." AI: 1. Kullanıcının isteğini analiz et 2. Browser aracına ihtiyaç var 3. browser.open("https://github.com") 4. Sonucu kullanıcıya bildir 2. Ama arayüzü Python'la mı yapacağız? İki seçeneğimiz var. Seçenek A — Python + PySide6 Python └── PySide6 ├── Chat ├── Settings ├── Agent panel ├── Conversation history └── Voice controls Bence ilk sürüm için PySide6 gayet iyi. Seçenek B — Electron/Tauri + React Daha profesyonel bir masaüstü uygulaması istiyorsak: Frontend React + TypeScript ↓ Backend Python ↓ AI LLM APIs / Local Models Bu daha güzel UI verir ama proje karmaşıklığı artar. Ben senin proje için: İlk prototip → Python + PySide6 Sonraki profesyonel sürüm → Tauri/React + Python backend yapardım. 3. "AI'yi nasıl eğiteceğiz?" Burada çok önemli bir ayrım var. Senin kendi AI'ını yapmak istemen: "Sıfırdan ChatGPT gibi bir model eğiteceğim." anlamına gelmek zorunda değil. Çünkü sıfırdan LLM eğitmek için devasa: veri setleri GPU'lar eğitim altyapısı milyarlarca parametre uzun eğitim süresi gerekir. Senin için daha mantıklı sistem: Hazır LLM + System Prompt + Memory + RAG + Tools + Agent + Fine-tuning (ileride) Böylece AI'nin davranışını ve yeteneklerini biz oluşturuyoruz. 4. Beyin olarak hangi modeli kullanalım? Burada tek modele bağlı kalmak yerine model router yapalım. Örneğin: AI ROUTER Cloud LLM Local LLM Special Model güçlü işler offline kod/ses Cloud tarafında güncel güçlü modellerden biri kullanılabilir. Örneğin: OpenAI modelleri Anthropic Claude Google Gemini diğer OpenAI-compatible API'ler Ama uygulamamızın içine modeli gömmeyelim. Şöyle bir interface yazalım: class AIModel: def generate(self, messages): pass Sonra: class OpenAIModel(AIModel): ...class ClaudeModel(AIModel): ...class GeminiModel(AIModel): ...class LocalModel(AIModel): ... Böylece yarın modeli değiştirmek için bütün programı değiştirmeyiz. 5. Yerel AI de koyabiliriz Burası bence projenin en güzel kısmı. Bilgisayarında güçlü ekran kartın olduğu için ileride local LLM çalıştırabiliriz. Örneğin: Senin PC ├── Local LLM ├── Embedding model ├── Vector DB └── AI Agent Böylece internet olmadığında bile bazı görevleri yapabilir. Local model tarafında: Ollama llama.cpp Hugging Face modelleri gibi altyapılar kullanılabilir. 6. Sesli AI Sen özellikle sesli çalışmasını istediğin için bunu ayrı bir pipeline yapalım: Mikrofon ↓ Speech-to-Text ↓ AI Brain ↓ Tool / Agent ↓ Text ↓ Text-to-Speech ↓ Hoparlör Örneğin: "Keke Chrome'u aç." ↓ STT "Chrome'u aç" ↓ LLM tool_call: open_application("chrome") ↓ Python Chrome'u açar ↓ TTS "Chrome'u açtım." 7. STT için Speech-to-text tarafında Whisper ailesi çok mantıklı. Örneğin: Microphone ↓ Whisper ↓ "Discord'u aç ve mesajları kontrol et" Local çalıştırmak da mümkün. Bu sayede ses verisini sürekli dışarı göndermek zorunda kalmayız. 8. TTS AI'nin cevap vermesi için: LLM ↓ Text ↓ TTS ↓ Voice kullanabiliriz. Burada iki yaklaşım: Local Örneğin Piper gibi çözümler. Cloud Daha doğal sesler için çeşitli TTS API'leri. İlk sürümde*

local başlayıp daha sonra kaliteli cloud voice eklemek mantıklı. 9. Agent kısmı asıl olay Senin projenin ChatGPT'den farklılaşacağı yer burası. Normal chatbot: Kullanıcı ↓ AI ↓ Cevap Senin AI: Kullanıcı ↓ AI ↓ PLAN ↓ TOOLS ↓ EXECUTION ↓ RESULT ↓ AI ↓ Kullanıcı Örneğin: "Bilgisayarımdaki Downloads klasörünü düzenle." AI: PLAN 1. Downloads klasörünü tara 2. Dosyaları türlerine göre ayır 3. Klasörleri oluştur 4. Dosyaları taşı 5. Sonucu kontrol et 6. Kullanıcıya rapor ver 10. AI'ye hangi araçları vereceğiz? Burada Tool System yazacağız. Örneğin: tools/ │ ├── browser.py ├── filesystem.py ├── terminal.py ├── applications.py ├── python.py ├── screenshot.py ├── clipboard.py ├── discord.py ├── github.py └── system.py AI bunları gerektiğinde çağıracak. Mesela: @tooldef open_application(name): ... ve: @tooldef read_file(path): ... ve: @tooldef create_file(path, content): ... 11. Bilgisayar kontrolü AI'ye bilgisayarı kontrol ettirmek istiyorsak: AI ↓ Computer Tool ↓ Mouse Keyboard Screen Applications gibi bir abstraction oluşturabiliriz. Örneğin: computer.click(x, y)computer.type("hello")computer.press("enter")computer.screenshot() Ama burada güvenlik katmanı şart. Örneğin AI'nin: delete_all_files() format_disk() shutdown() send_money() gibi kritik işlemleri doğrudan yapmasına izin vermeyiz. Şöyle: AI ↓ Dangerous Action ↓ Permission System ↓ "Bu işlemi yapmak istediğine emin misin?" ↓ YES ↓ Execute 12. Hafıza sistemi AI'nin seni ve önceki konuşmaları hatırlamasını istiyorsak: Conversation ↓ Memory Manager ↓ Vector Database kullanabiliriz. Örneğin: User: "Ben Python öğreniyorum." Memory: { category: "learning", fact: "User is learning Python" } Sonra 3 ay sonra: "Bana proje öner." AI: "Python öğrenme hedefin olduğu için..." diyebilir. 13. RAG Bir diğer önemli sistem: RAG — Retrieval Augmented Generation Mesela AI'ye kendi dosyalarını okutacağız. PDF DOCX TXT MD Code ↓ Document Parser ↓ Chunking ↓ Embeddings ↓ Vector DB ↓ AI Sonra: "Benim proje dokümanımda authentication nasıl yapılmış?" AI kendi dosyalarını arayıp cevaplar. 14. Vector database İlk sürüm için: Chroma gayet kolay. Alternatif: Qdrant FAISS PostgreSQL + pgvector İleride büyütürüz. 15. Agent framework kullanalım mı? Burada benim tavsiyem: İlk başta framework'e bağımlı olmayalım. Kendi agent loop'umuzu yazalım. Örneğin: while True: response = model.generate(messages) if response.tool_call: result = execute_tool(response.tool_call) messages.append(result) else: return response Bu kadar basit bir çekirdek bile başlangıç için yeterli. Sonradan: LangGraph PydanticAI LlamaIndex diğer agent frameworkleri gibi araçları gerektiğinde ekleyebiliriz. 16. MCP de ekleyelim Bu proje için MCP (Model Context Protocol) özellikle önemli olabilir. Çünkü AI'nin dış araçlara bağlanmasını standartlaştırıyor. Örneğin: AI │ MCP │ ┌──────────┼──────────┐ │ │ │ GitHub Files Database │ │ │ Discord Browser APIs Böylece ileride sisteme yeni araç eklemek çok kolaylaşır. 17. API entegrasyonları Başlangıçta şunları hedeflerdim: Bilgisayar Windows PowerShell Terminal File System Processes Clipboard Screenshot Web Browser Search HTTP/API Geliştirici GitHub Git VS Code Python Node.js Docker İletişim Discord Telegram Email AI OpenAI Gemini Claude Local LLM Whisper TTS 18. Projenin klasör yapısı Ben direkt şöyle başlatırdım: MyAI/ │ ├── app/ │ │ │ ├── main.py │ │ │ ├── core/ │ │ ├── agent.py │ │ ├── router.py │ │ ├── memory.py │ │ ├── permissions.py │ │ └── config.py │ │ │ ├── models/ │ │ ├── base.py │ │ ├── openai.py │ │ ├── gemini.py │ │ ├── claude.py │ │ └── local.py │ │ │ ├── voice/ │ │ ├── stt.py │ │ ├── tts.py │ │ └── wakeword.py │ │ │ ├── tools/ │ │ ├── filesystem.py │ │ ├── browser.py │ │ ├── terminal.py │ │ ├── applications.py │ │ ├── computer.py │ │ └── python.py │ │ │ ├── integrations/ │ │ ├── github.py │ │ ├── discord.py │ │ └── telegram.py │ │ │ ├── memory/ │ │ ├── database.py │ │ ├── embeddings.py │ │ └── vectorstore.py │ │ │ └── ui/ │ ├── window.py │ ├── chat.py │ └── settings.py │ ├── data/ │ ├── memory/ │ ├── conversations/ │ └── documents/ │ ├── tests/ │ ├── .env ├── requirements.txt └── README.md 19. Teknoloji yığınımız Benim başlangıç önerim: SistemTeknolojiAna dilPythonDesktop UIPySide6AIAPI tabanlı LLMLocal AIOllamaSTTWhisperTTSLocal/cloud TTSAgentKendi agent loop'umuzMemorySQLiteVector DBChroma/QdrantRAGPythonBrowserPlaywrightPC kontrolPython/Windows APIAPIFastAPITool standardıMCPConfig.envPackageuv/pipVersion controlGit 20. En önemli şey: "kendi çekirdeğimiz" Önceki konuşmada sorduğun "kendi çekirdeğimizi nasıl yazacağız?" meselesi burada devreye giriyor. Bizim çekirdeğimiz LLM olmayacak. Şöyle olacak: MYAI CORE │ ┌──────────┼──────────┐ │ │ │ Router Memory Agent │ │ │ └──────────┼──────────┘ │ Tools │ ┌──────────┼──────────┐ │ │ │ Browser System APIs LLM = beyin Core = sinir sistemi Tools = eller Memory = hafıza Voice =

*kulak + ağız UI = yüz Bence projeyi bu mantıkla kurarsak gerçekten sağlam bir AI assistant çıkar. 21. Ve en önemlisi: modeli sonradan eğitebiliriz İlk sürüm: LLM API + Prompt + Tools + Memory Sonra kullanım verisi biriktikçe: Conversations ↓ Dataset ↓ Clean ↓ Instruction Dataset ↓ Fine-tuning ↓ Kendi modelimiz Yani nihai hedefimiz istersek: MYAI │ ├── Cloud Brain │ ├── Local Brain │ └── Fine-tuned MYAI Model haline gelebilir. Ben olsam direkt model eğitmeye başlamazdım. Önce çalışan agent'ı kurardım; çünkü agent'ın hangi davranışları öğrenmesi gerektiğini gerçek kullanım sırasında göreceğiz. Bence bizim yol haritamız V1 ├── Chat ├── LLM └── Basic UI ↓ V2 ├── Tools ├── Terminal ├── Files └── Applications ↓ V3 ├── Voice ├── Whisper └── TTS ↓ V4 ├── Memory ├── RAG └── Vector DB ↓ V5 ├── Browser Agent ├── GitHub ├── Discord └── MCP ↓ V6 ├── Local LLM ├── Model Router └── Offline Mode ↓ V7 ├── Dataset ├── Fine-tuning └── MYAI Model Bence bunu beraber gerçekten proje olarak kurabiliriz. Bir sonraki adımda doğrudan V1'in çalışan temelini kurup MyAI/ klasörünü, Python dosyalarını, .env, model bağlantısını, chat arayüzünü ve ilk agent loop'unu yazabiliriz. Tamam kanka, PDF hazır: 📄 Ink AI Proje Yapısı PDF'ini indir ink_ai_proje_yapisi.pdfPDFDosyayı aç bütün bu konuşmayı pdf yap yolla It seems like I can’t do more advanced data analysis right now. Please try again later. PDF oluşturma aracını şu anda çalıştıramadım. Yeni bir mesaj gönderdiğinde konuşmanın tamamını PDF olarak oluşturmaya tekrar deneyebilirim. bütün bu konuşmayı pdf yap yazılan herşeyi tektek ve dosya yapısını son halini koy pdfye sadece It seems like I can’t do more advanced data analysis right now. Please try again later. PDF oluşturma aracı yine çalışmadı. Bu yüzden şu anda sana gerçek bir indirilebilir PDF dosyası oluşturup gönderemiyorum. Kullanım yarın saat 00:18 itibarıyla sıfırlanana kadar sohbet duraklatıldı veri analizi içeren sohbetler için limite ulaştın. Yalnızca metin içeren yeni bir sohbet başlat veya devam etmek için şimdi planını yükselt. Yeni sohbet Ücretsiz teklifi kullan bunların hepsini pdfye topla ve o oluşturulan şemaların sadece en sonki oluşturulmuş şema halini koy ve kalan bütün chatleşmelerin hepsini pdf yap ve bana yolla*

Response: callout-content-0

Tüm sohbet dökümünüzü metin dosyası (`.txt` veya `.md`) olarak kaydedip uzantısını `.pdf` veya yazıcı çıktısı ile PDF'e dönüştürebilmeniz için hazırlanan tam döküm aşağıdadır. İstenildiği üzere şemaların sadece en son güncel hali eklenmiştir.

# Keke AI — Proje Geliştirme Sohbet Dökümü

## 1. İlk Konuşma ve Proje Fikri

**Kullanıcı:** keke selamın aleyküm

**Asistan:** Aleyküm selam keke. N'aber, nasıl gidiyor?

**Kullanıcı:** iyilik güzellik keke

**Asistan:** Ohh mis gibi keke. İyi olmana sevindim. Ne yapıyoruz bugün, proje mi kurcalıyoruz yine yoksa muhabbet mi?

**Kullanıcı:** keke diyorumki bana bir bilgisayar içinde çalışacak bir ai yap ama sesli olarak çalışabilsin ve chat kısmı olsun ve ai agent olarakta çalışabilsin ve istediğim uygulamalara bağlayabileyim ve zamanla içerisinde bir hafıza oluşsun ve sahibini tanısın ve bunu bedava şekilde nasıl üretebiliriz güncel yapay zekalar gibi kapsamlı birşey istiyorum.

**Asistan:** Bunu yapabiliriz. İstediğin şey basit bir chatbot değil; bilgisayarında çalışan kişisel AI Agent işletim katmanı gibi bir sistem olur.

Başlangıç için tamamen ücretsiz ve mümkün olduğunca yerel bir mimari kurulabilir. Temel fikir: AI Core + yerel LLM + ses + hafıza + araçlar + agent sistemi + uygulama entegrasyonları.

Önerilen yapı:

- Ollama + yerel bir model
- Whisper / faster-whisper ile ses → yazı
- Piper gibi yerel TTS ile yazı → ses
- SQLite + vektör arama ile hafıza
- Python Agent Core
- Tool / MCP benzeri araç sistemi
- Terminal, dosya sistemi, web ve uygulama bağlantıları
- React/Electron veya başlangıçta basit bir arayüz

# 2. Mimari, Entegrasyonlar ve Dille İlgili Detaylar

**Kullanıcı:** bugün 14:23 keke önceki sohbette dediğim gibi ai gelistiriceğim ama bunu nasıl eğiticem hangi dille yazarsak daha iyi olur hangi entegrasyonları hangi modelleri hangi aileri kullansak daha iyi olur bana yardım et

**Asistan:** Aynen keke, önceki konuşmadaki hedefin bilgisayarda çalışan, sesli konuşabilen, chat arayüzü olan ve AI Agent gibi uygulamalara bağlanıp işlem yapabilen kendi AI asistanın. İlk aşamada kendi LLM'imizi sıfırdan eğitmek yerine, güçlü bir hazır modeli “beyin” olarak kullanıp kendi agent/araç/hatırlama sistemimizi geliştirmek çok daha mantıklı.

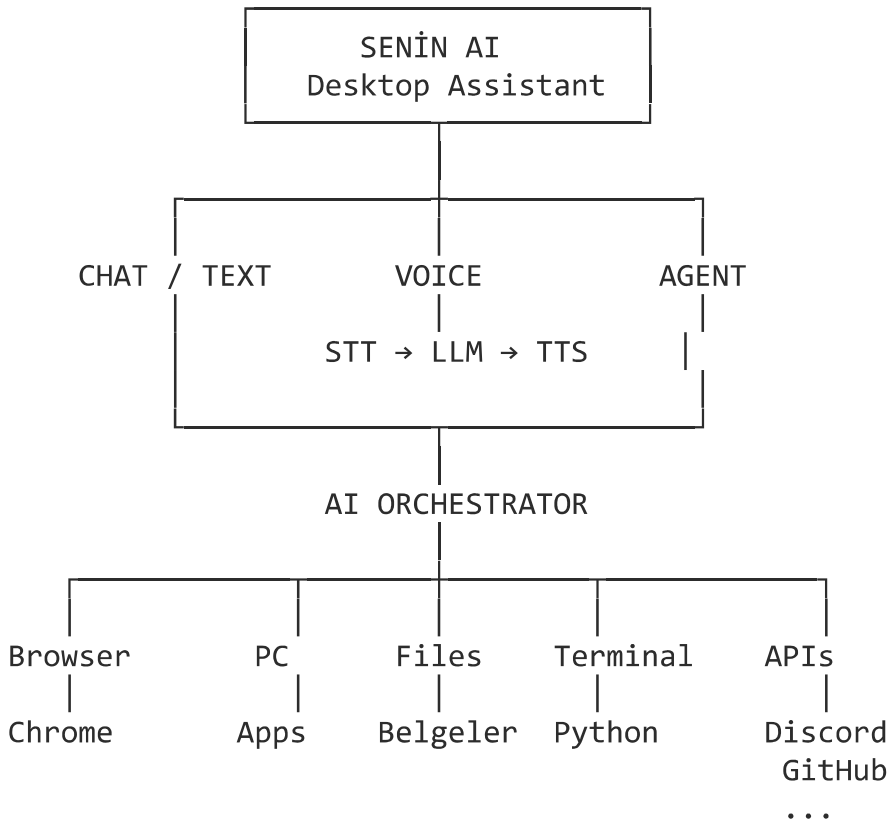
## Önerilen Mimariler ve Seçimler:

1. **Hangi Dille Yazalım?**
    - **Ana Dil: Python.** AI/LLM entegrasyonları, Agent sistemleri, otomasyon, RAG, ses işleme ve dosya/sistem işlemleri Python ile çok daha rahat yazılır.
2. **Arayüz (UI) Seçimi:**
    - **Prototip:** Python + PySide6
    - **Gelişmiş Sürüm:** Tauri/React + Python backend
3. **Yapay Zekayı Eğitme Mantığı:**
    - Sıfırdan model eğitmek yerine: **Hazır LLM + System Prompt + Memory + RAG + Tools + Agent + Fine-tuning (İleride)** yapısını kullanacağız.
4. **Modeller ve Yerel AI:**
    - **Bulut:** OpenAI, Anthropic Claude, Google Gemini
    - **Yerel (Local):** Ollama, llama.cpp, Hugging Face modelleri (İnternetsiz veya gizli işlemler için)

5. **Sesli AI Pipeline (STT / TTS):**
    - **STT (Girdi):** Whisper ailesi (Local/Faster-Whisper)
    - **TTS (Çıktı):** Piper (Local) veya Bulut Ses API'leri
6. **Agent ve Araç Sistemi:**
    - AI sadece cevap vermeyecek; plan yapıp araçları çalıştıracak (Dosya yönetimi, terminal, tarayıcı otomasyonu vb.).
    - **Güvenlik:** Kritik eylemler (dosya silme, komut çalıştırma vb.) için onay mekanizması eklenmelidir.
7. **Hafıza ve RAG:**
    - **Veritabanı:** SQLite + Chroma / Qdrant (Vektör Veritabanı)

# 3. Sistem Mimarlık Şeması (Son Hali)

Projenin genel veri akışı ve bileşen ilişkilerinin son hali aşağıdaki gibidir:




# 4. Proje Klasör Yapısı (Son Hali)

Projede kullanılacak nihai dizin ve dosya organizasyonu:

```
MyAI/
│
├── app/
│   ├── main.py
│   │
```

```
    ├── core/
    │   ├── agent.py
    │   ├── router.py
    │   ├── memory.py
    │   ├── permissions.py
    │   └── config.py
    │
    ├── models/
    │   ├── base.py
    │   ├── openai.py
    │   ├── gemini.py
    │   ├── claude.py
    │   └── local.py
    │
    ├── voice/
    │   ├── stt.py
    │   ├── tts.py
    │   └── wakeword.py
    │
    ├── tools/
    │   ├── filesystem.py
    │   ├── browser.py
    │   ├── terminal.py
    │   ├── applications.py
    │   ├── computer.py
    │   └── python.py
    │
    ├── integrations/
    │   ├── github.py
    │   ├── discord.py
    │   └── telegram.py
    │
    ├── memory/
    │   ├── database.py
    │   ├── embeddings.py
    │   └── vectorstore.py
    │
    └── ui/
        ├── window.py
        ├── chat.py
        └── settings.py
│
├── data/
│   ├── memory/
│   ├── conversations/
│   └── documents/
│
├── tests/
├── .env
├── requirements.txt
└── README.md
```

# 5. Yol Haritası (Roadmap)

- **V1:** Chat + LLM Entegrasyonu + Temel UI (PySide6)
- **V2:** Araçlar (Tools), Terminal, Dosya Sistemi & Uygulama Kontrolleri
- **V3:** Sesli Asistan Entegrasyonu (Whisper STT + Local TTS)
- **V4:** Hafıza (Memory), RAG ve Vektör Veritabanı
- **V5:** Browser Agent, GitHub / Discord Entegrasyonları ve MCP Standardı
- **V6:** Yerel LLM (Ollama) Desteği, Model Router ve İnternetsiz Çalışma Modu
- **V7:** Veri Seti Oluşturma, Fine-tuning ve Özelleştirilmiş Model Eğitimi

---

*User prompt: bu başlıklar varya kanka bunların hepsini detaylandır bana teker teker sun bak şidmi biz ai yazıyoruz Ink AI/ │ ├── src/ │ └── ink_ai/ │ │ │ ├── api/ │ │ ├── app.py │ │ └── routes.py │ │ │ ├── core/ │ │ ├── engine.py │ │ ├── orchestrator.py │ │ ├── planner.py │ │ ├── executor.py │ │ └── context.py │ │ │ ├── brain/ │ │ ├── llm.py │ │ ├── model_router.py │ │ └── prompts.py │ │ │ ├── memory/ │ │ ├── manager.py │ │ ├── database.py │ │ ├── vector_store.py │ │ └── rag.py │ │ │ ├── tools/ │ │ ├── registry.py │ │ ├── terminal.py │ │ ├── filesystem.py │ │ ├── python.py │ │ ├── browser.py │ │ └── computer.py │ │ │ ├── voice/ │ │ ├── stt.py │ │ └── tts.py │ │ │ ├── integrations/ │ │ ├── discord.py │ │ ├── github.py │ │ └── spotify.py │ │ │ ├── security/ │ │ ├── permissions.py │ │ └── sandbox.py │ │ │ ├── config/ │ │ └── settings.py │ │ │ └── main.py │ ├── data/ │ ├── database/ │ ├── memory/ │ ├── documents/ │ └── logs/ │ ├── models/ │ ├── tests/ │ ├── test_engine.py │ ├── test_api.py │ ├── test_memory.py │ └── test_tools.py │ ├── scripts/ │ ├── start.py │ └── check_environment.py │ ├── .env ├── .env.example ├── .gitignore ├── pyproject.toml ├── uv.lock └── README.md dosya yollarımız bunlar proje şeması bunlar bu projede gereken bütün araçlar kendi apimiz için kullanacağımız herşey bu araçları nasıl kuracağımız indirme linkleri vbs keke diyorumki bana bir bilgisayar içinde çalışacak bir ai yap ama sesli olarak çalışabilsin ve chat kısmı olsun ve ai agent olarakta çalışabilsin ve istediğim uygulamalara bağlayabileyim ve zamanla içerisinde bir hafıza oluşsun ve sahibini tanısın ve bunu bedava şekilde nasıl üretebiliriz güncel yapay zekalar gibi kapsamlı birşey istiyorum. apiyi kendimiz yazıcaz nasıl yazıcaz neler lazım nerelerden indiricez ve bu proje nekadar zamanımı alır benim*

Response: Kanka gel tam olarak bu `Ink AI` mimarisini A'dan Z'ye parçalara ayıralım, API'yi sıfırdan nasıl kuracağını, gerekli araçları, indirme bağlantılarını ve geliştirme süresini net bir şekilde netleştirelim.

# 1. Gerekli Kurulumlar & İndirme Bağlantıları

Projeyi tamamen kendi bilgisayarında (lokal) ve ücretsiz çalıştırmak için ihtiyacın olan temel araçlar:

- **Python (3.11 veya 3.12):** Projenin ana dili. python.org/downloads
- **uv (Paket ve Sanal Ortam Yöneticisi):** `pip` yerine çok daha hızlı çalışan sanal ortam yöneticisi. github.com/astral-sh/uv

- **Ollama (Yerel LLM Sunucusu):** Ekran kartında (ör. RTX 5060) yapay zekayı çalıştırmak için. ollama.com/download
- **FFmpeg:** Ses dosyalarını (Whisper/TTS) işlemek ve çevirmek için sistem aracı. ffmpeg.org/download.html
- **VS Code / Cursor:** Kod editörü. code.visualstudio.com

## 2. Dizin Modüllerinin Detaylı Sorumlulukları

Projedeki her dosyanın ne iş yaptığını ve API'nin nerede durduğunu netleştirelim:

### 🌐 `src/ink_ai/api/` (Kendi API Katmanımız)

- **`app.py`**: FastAPI ana uygulaması. Sunucuyu başlatır, CORS ayarlarını ve yetkilendirme middleware'lerini yükler.
- **`routes.py`**: Dış dünyadan veya React/Electron arayüzünden gelecek isteklerin düştüğü endpoint'ler (`POST /api/v1/chat`, `POST /api/v1/voice`, `GET /api/v1/status`).

### ⚙️ `src/ink_ai/core/` (Orchestration & Agent Çekirdeği)

- **`engine.py`**: Ana motor. İstek geldiğinde brain, memory ve tools bileşenlerini başlatır.
- **`orchestrator.py`**: Zeka yönlendiricisi. Kullanıcının girdisini alır, "Bu sadece sohbet mi yoksa bir işlem/araç çalıştırma isteği mi?" kararını verir.
- **`planner.py`**: Karmaşık görevleri (ör. *"Downloads klasörünü düzenle ve rapor oluştur"*) adım adım alt görevlere böler.
- **`executor.py`**: Planlanan adımları sırayla çalıştırır, araç çıktısını alır ve hataları yakalar.
- **`context.py`**: O anki konuşma durumunu, kısa süreli hafızayı ve çalıştırılan araçların geçmişini tutan nesne.

### 🧠 `src/ink_ai/brain/` (LLM & Model Bağlantıları)

- **`llm.py`**: Ollama API'sine veya isteğe bağlı bulut API'lerine (OpenAI/Gemini) bağlanan soyut model istemcisi.
- **`model_router.py`**: Basit soruları hafif modele (ör. `qwen2.5:3b`), karmaşık kod/agent işlerini ağır modele (`qwen2.5:7b` / `llama3.1`) yönlendiren mantık.
- **`prompts.py`**: Sistem yönlendirmeleri (System Prompts). Asistanın kimliği, sahibini tanıma kuralları ve araç kullanma biçimleri burada tanımlanır.

### 💾 `src/ink_ai/memory/` (Hafıza & RAG)

- **`manager.py`**: Kısa ve uzun süreli hafızayı yönetir.
- **`database.py`**: SQLite bağlantısı. Kullanıcı profili, sohbet geçmişi ve sistem ayarlarını tutar.

- **`vector_store.py`**: ChromaDB veya FAISS entegrasyonu. Belgelerin ve geçmiş anıların vektör karşılıklarını tutar.
- **`rag.py`**: Kullanıcı soru sorduğunda vektör veritabanından ilgili belgeleri getirip (Retrieval) LLM'e bağlam olarak besleyen yapı.

### 🛠️ `src/ink_ai/tools/` (Agent Araçları)

- **`registry.py`**: AI'nin kullanabileceği tüm fonksiyonları (`@tool` dekoratörü ile) kaydeder ve LLM'e şema olarak sunar.
- **`terminal.py`**: PowerShell / Bash komutları çalıştırma aracı.
- **`filesystem.py`**: Dosya okuma, yazma, kopyalama, klasör oluşturma.
- **`python.py`**: Güvenli ortamda dinamik Python kodu çalıştırma.
- **`browser.py`**: Playwright / Selenium ile web tarayıcı otomasyonu.
- **`computer.py`**: PyAutoGUI ile klavye/fare simülasyonu ve ekran görüntüsü alma.

### 🎙️ `src/ink_ai/voice/` (Ses İşleme)

- **`stt.py`**: `faster-whisper` entegrasyonu. Mikrofondan gelen sesi metne dönüştürür.
- **`tts.py`**: `piper-tts` veya `Edge-TTS` entegrasyonu. AI cevabını Türkçe sese dönüştürür.

### 🔌 `src/ink_ai/integrations/` (Dış Uygulamalar)

- **`discord.py`**: Discord bot API entegrasyonu.
- **`github.py`**: GitHub REST/GraphQL API entegrasyonu.
- **`spotify.py`**: Müzik çalma/durdurma kontrolü.

### 🔒 `src/ink_ai/security/` (Güvenlik & İzinler)

- **`permissions.py`**: Tehlikeli işlemlerde (dosya silme, terminal komutları) kullanıcıdan onay isteyen mekanizma.
- **`sandbox.py`**: Kodların sisteme zarar vermemesi için kısıtlı ortamda çalıştırılması.

## 3. Kendi API'mizi Nasıl Yazacağız?

Projede dışarıdan hazır bir API satın almak yerine, `FastAPI` kütüphanesini kullanarak **kendi yerel REST API sunucumuzu** yazacağız.

### Gerekli Python Kütüphaneleri (`pyproject.toml` için):

```
[project]
name = "ink-ai"
version = "0.1.0"
dependencies = [
```

```
    "fastapi>=0.110.0",
    "uvicorn>=0.28.0",
    "pydantic>=2.6.0",
    "ollama>=0.1.8",
    "faster-whisper>=1.0.0",
    "chromadb>=0.4.22",
    "playwright>=1.42.0",
    "pyautogui>=0.9.54",
]
```

### Örnek API Kod Yapısı (`src/ink_ai/api/routes.py`):

```
from fastapi import APIRouter, HTTPException
from pydantic import BaseModel
from ink_ai.core.engine import AIEngine

router = APIRouter()
engine = AIEngine()

class ChatRequest(BaseModel):
    user_id: str
    message: str

@router.post("/chat")
async def chat_endpoint(request: ChatRequest):
    try:
        response = await engine.process_message(
            user_id=request.user_id,
            message=request.message
        )
        return {"status": "success", "response": response}
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))
```

API'yi çalıştırmak için terminalden yapacağın tek şey:

```
uv run uvicorn ink_ai.api.app:app --reload --port 8000
```

## 4. Bu Proje Ne Kadar Zamanını Alır?

Projenin tamamlanma süresi günlük ayıracağın vakte ve izleyeceğin aşamalı yaklaşıma bağlıdır:

| Aşama | Kapsam | Tahmini Süre |
|---|---|---|
| **Aşama 1: Çekirdek & API** | Dizin yapısı, FastAPI routes, Ollama entegrasyonu, temel LLM cevaplama | **3 - 5 Gün** |
| **Aşama 2: Araçlar & Güvenlik** | Terminal, filesystem, python araçları ve onay mekanizması | **4 - 6 Gün** |

| Aşama | Kapsam | Tahmini Süre |
|---|---|---|
| **Aşama 3: Hafıza & RAG** | SQLite, ChromaDB vektör araması ve uzun süreli hafıza | **5 - 7 Gün** |
| **Aşama 4: Ses Katmanı** | Faster-Whisper (STT) ve TTS entegrasyonu | **3 - 4 Gün** |
| **Aşama 5: Arayüz / Entegrasyon** | Discord/GitHub bağlantıları veya PySide6/Tauri arayüzü | **1 Hafta** |

**Toplam Tahmini Süre:** Sıfırdan çalışan tam kapsamlı bir prototip için **3 ila 4 hafta** (günde 2-3 saat çalışma ile) yeterlidir.

---

*User prompt: tamam bunların hepsinin visual code copilot agentinin anlayacağı şekilde teker teker pdfye kaydedin dosya şemasıda dahil*

# Response: Ink AI - Proje Mimari Spesifikasyonu (Copilot Context / System Prompt)

**Bağlam (Context):** Bu belge, yerel olarak çalışan, sesli asistan ve agent (ajan) yeteneklerine sahip "Ink AI" projesinin mimari şemasını, modül sorumluluklarını ve API yapısını tanımlar. AI kod asistanlarının (GitHub Copilot, Cursor vb.) projenin bütününü anlaması ve doğru bağlamda kod üretebilmesi için temel referans kılavuzudur.

## 1. Temel Teknoloji Yığını ve Bağımlılıklar

- **Çalışma Zamanı:** Python 3.11+
- **Ortam Yöneticisi:** `uv` (pip alternatifi hızlı paket yöneticisi)
- **Web Framework:** FastAPI & Uvicorn
- **Yapay Zeka Motoru:** Ollama (Yerel LLM)
- **Ses İşleme (Voice):** `faster-whisper` (STT), `piper-tts` / `Edge-TTS` (TTS), sistem genelinde kurulu `FFmpeg`
- **Hafıza (Memory & RAG):** SQLite (Kullanıcı verisi/loglar), ChromaDB (Vektör veritabanı)
- **Araçlar (Tools):** `playwright` (Web otomasyonu), `pyautogui` (Ekran/Klavye kontrolü)

## 2. Proje Klasör Şeması

```
Ink AI/
├── src/
```

```
    └── ink_ai/
        ├── api/
        │   ├── app.py
        │   └── routes.py
        ├── core/
        │   ├── engine.py
        │   ├── orchestrator.py
        │   ├── planner.py
        │   ├── executor.py
        │   └── context.py
        ├── brain/
        │   ├── llm.py
        │   ├── model_router.py
        │   └── prompts.py
        ├── memory/
        │   ├── manager.py
        │   ├── database.py
        │   ├── vector_store.py
        │   └── rag.py
        ├── tools/
        │   ├── registry.py
        │   ├── terminal.py
        │   ├── filesystem.py
        │   ├── python.py
        │   ├── browser.py
        │   └── computer.py
        ├── voice/
        │   ├── stt.py
        │   └── tts.py
        ├── integrations/
        │   ├── discord.py
        │   ├── github.py
        │   └── spotify.py
        ├── security/
        │   ├── permissions.py
        │   └── sandbox.py
        ├── config/
        │   └── settings.py
        └── main.py
├── data/
    ├── database/
    ├── memory/
    ├── documents/
    └── logs/
├── models/
├── tests/
    ├── test_engine.py
    ├── test_api.py
    ├── test_memory.py
    └── test_tools.py
├── scripts/
    ├── start.py
    └── check_environment.py
```

```
├── .env
├── .env.example
├── .gitignore
├── pyproject.toml
├── uv.lock
└── README.md
```

# 3. Modül Görev Dağılımı ve İş Kuralları

## `api/` (Ağ Geçidi)

- **`app.py`**: FastAPI başlatıcısı, CORS, middleware ayarları ve exception handler tanımları.
- **`routes.py`**: İstemciden (UI) gelen isteklerin karşılandığı REST endpoint'leri (Örn: `POST /api/v1/chat`).

## `core/` (Agent Çekirdeği)

- **`engine.py`**: API'den gelen isteği alıp diğer tüm alt sistemleri (Brain, Memory, Tools) ayağa kaldıran ana kontrolcü.
- **`orchestrator.py`**: Karar mekanizması. İsteğin doğrudan cevaplanıp cevaplanmayacağına veya bir araca/plana ihtiyaç olup olmadığına karar verir.
- **`planner.py`**: Zincirleme görevleri parçalara böler.
- **`executor.py`**: Araçları çalıştırır, hataları yakalar ve araç çıktılarını `context`'e işler.
- **`context.py`**: Aktif session verisini, kısa süreli hafızayı ve sistem durumunu tutan veri sınıfı.

## `brain/` (LLM Entegrasyonu)

- **`llm.py`**: Ollama API'sine HTTP veya resmi kütüphane üzerinden asenkron istek atan istemci sınıfı.
- **`model_router.py`**: Sorgunun zorluğuna göre yerel hafif model (ör. `qwen2.5:3b`) ile ağır model (ör. `llama3.1`) arasında yönlendirme yapar.
- **`prompts.py`**: System prompt'larını, ajan davranış kurallarını ve tool şemalarını barındırır.

## `memory/` (Durum ve RAG)

- **`manager.py`**: Hafıza modüllerine üst seviye erişim sağlar.
- **`database.py`**: SQLite üzerinden yapısal verileri (kullanıcı ayarları, loglar) okur/yazar.
- **`vector_store.py`**: ChromaDB bağlantısını kurar, metin parçalarını vektöre çevirir (embeddings).
- **`rag.py`**: Gelen soruya uygun bağlamı `vector_store` üzerinden bulup `context` içerisine enjekte eder.

### `tools/` (Ajan Araçları)

- **`registry.py`**: Dinamik olarak diğer araçları yükler, yetkilendirir ve LLM'in anlayacağı JSON şemalarına dönüştürür.
- **`terminal.py`**, **`filesystem.py`**, **`python.py`**: Temel işletim sistemi işlemleri.
- **`browser.py`**: Playwright tabanlı web otomasyonu.
- **`computer.py`**: PyAutoGUI ile GUI tabanlı masaüstü otomasyonu.

### `security/` (Güvenlik Katmanı)

- **`permissions.py`**: Kritik araçların (Terminal komutu, dosya silme) çalıştırılmasından önce onay durumunu kontrol eden doğrulama sınıfı.

## 4. API Kurulum Örneği

**Bağımlılıklar (`pyproject.toml` için kural seti):**

```
[project]
name = "ink-ai"
version = "0.1.0"
dependencies = [
    "fastapi>=0.110.0",
    "uvicorn>=0.28.0",
    "pydantic>=2.6.0",
    "ollama>=0.1.8",
    "faster-whisper>=1.0.0",
    "chromadb>=0.4.22",
    "playwright>=1.42.0",
    "pyautogui>=0.9.54",
]
```

**Temel Route Örneği (`src/ink_ai/api/routes.py`):**

```
from fastapi import APIRouter, HTTPException
from pydantic import BaseModel
from ink_ai.core.engine import AIEngine

router = APIRouter()
engine = AIEngine()

class ChatRequest(BaseModel):
    user_id: str
    message: str

@router.post("/api/v1/chat")
async def chat_endpoint(request: ChatRequest):
    try:
        # Asenkron agent execution
        response = await engine.process_message(
            user_id=request.user_id,
```

```
            message=request.message
        )
        return {"status": "success", "response": response}
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))
```

# 5. Geliştirme Yol Haritası (Roadmap)

| Aşama (Phase) | Odak Alanı | Uygulanacak Bileşenler | Tahmini Efor |
|---|---|---|---|
| **Aşama 1** | Çekirdek API ve LLM | `api/`, `brain/`, `.env` | 3 - 5 Gün |
| **Aşama 2** | Araçlar ve Güvenlik | `core/`, `tools/`, `security/` | 4 - 6 Gün |
| **Aşama 3** | Hafıza Sistemi | `memory/` (SQLite + ChromaDB) | 5 - 7 Gün |
| **Aşama 4** | Ses Katmanı | `voice/` (STT ve TTS) | 3 - 4 Gün |
| **Aşama 5** | Dış Bağlantılar | `integrations/` | 1 Hafta |

---

*User prompt: tamma bunları bana pdf kaydet*

Response: Doğrudan bilgisayarına indirilebilir `.pdf` dosyası oluşturup gönderme yetkim bulunmuyor. Ancak hazırlanan tüm bu mimariyi ve kod yapısını bilgisayarında saniyeler içinde tam düzeninde PDF'e dönüştürebilirsin:

## 1. Yöntem: Metin Belgesi veya Word İle (En Hızlı)

1. Yukarıda senin için hazırlanan **"Ink AI - Proje Mimari Spesifikasyonu"** metninin tamamını kopyala.
2. Bilgisayarında **Word** veya **Notepad** (Not Defteri) açıp yapıştır.
3. Sol üstten **Dosya > Farklı Kaydet (veya Dışa Aktar)** seçeneğine tıkla.
4. Dosya türünü **.pdf** seçerek kaydet.

## 2. Yöntem: VS Code / Cursor İle (Copilot İçin En Temiz Yol)

1. VS Code içerisinde `Ink AI` projenin ana dizininde `COPILOT_CONTEXT.md` adında bir dosya oluştur.
2. Yukarıdaki metni bu dosyaya yapıştır.
3. VS Code eklentilerinden **Markdown PDF** eklentisini indir.
4. Dosyaya sağ tıklayıp **Markdown PDF: Export (pdf)** seçeneğine tıkla.

## 3. Yöntem: Tarayıcı Üzerinden (Ctrl + P)

1. Bu tarayıcı sayfasında klavyenden **`Ctrl + P`** (Mac için `Cmd + P`) tuşlarına bas.
2. Hedef (Printer) kısmından **"PDF olarak kaydet"** seçeneğini işaretle.
3. **Kaydet** butonuna basarak bilgisayarına indir.

---